

Guion — Presentación Final rentFlowMax

Duración objetivo: ≈ 10 minutos · App Android de gestión de propiedades en renta

Nota: Reemplaza [Integrante 1/2/3/4] con los nombres reales y ajusta los “owners” según corresponda. Tiempos sugeridos entre paréntesis.

1. Apertura (0:45)

[Integrante 1]: Buenas tardes. Somos [Integrante 1], [Integrante 2], [Integrante 3] y [Integrante 4], y les presentamos **rentFlowMax**, una aplicación Android para la gestión integral de propiedades en renta.

Problema: Los pequeños arrendadores en México llevan el control de propiedades, inquilinos, contratos y pagos en hojas de cálculo o papel. Esto genera contratos vencidos sin detectar, pagos duplicados, cobros olvidados y nula trazabilidad del mantenimiento. **rentFlowMax centraliza todo en el celular**, en español y con moneda MXN.

2. Recorrido funcional — Demo en vivo (2:30)

[Integrante 2]: Recorramos el flujo desde la perspectiva del arrendador.

1. **Dashboard** — al abrir la app se ve el resumen: propiedades totales, rentadas, disponibles, contratos activos y pagos del mes.
2. **Propiedades** — registro una propiedad nueva (nombre, dirección, renta, depósito).
3. **Inquilinos** — doy de alta un inquilino con sus datos de contacto.
4. **Contratos** — creo el contrato vinculando propiedad + inquilino, con fechas y monto. Si intento crear un contrato sin propiedades, la app **me guía con un diálogo** al formulario de propiedades — UX de navegación guiada.
5. **Pagos** — registro el pago mensual; la app valida que no haya duplicado para el mismo periodo.
6. **Mantenimiento** — registro una solicitud asociada a la propiedad, cambio su estado (pendiente → en proceso → completado) con su costo.
7. **Borrado seguro** — si intento borrar una propiedad con contrato activo, se bloquea; si borro un contrato, advierte que se eliminarán pagos en cascada.

3. Estructura del proyecto y MVVM (1:30)

[Integrante 3]: Mostramos Android Studio. El paquete `com.example.rentflowmax` está dividido en tres capas claras:

- **Capa de datos (data/)**
 - `model/` — entidades Room: `Property` , `Tenant` , `Contract` , `Payment` , `Maintenance` .
 - `dao/` — interfaces DAO con queries SQL.
 - `db/AppDatabase.kt` — base de datos Room + `Converters` .
 - `repository/` — un repositorio por entidad, expone `Flow` .
- **Capa de presentación / UI (ui/)**
 - **View:** `Fragment` s (List, Detail, Form) + layouts XML con `ViewBinding`.
 - **ViewModel:** uno por pantalla, expone `StateFlow` , contiene la lógica de validación y orquestación.
 - `common/ViewModelFactory.kt` — DI manual.
- **Aplicación:** `RentFlowMaxApp.kt` instancia la base de datos y los repositorios (sin Hilt — DI manual).

Navegación: single-activity con Navigation Component (`nav_graph.xml`).

4. Tecnologías integradas — un ejemplo de código por cada una (1:30)

[Integrante 4]: Estas son las tecnologías clave y dónde verlas en el código:

Tecnología	Para qué	Dónde verlo
Room + KSP	Persistencia local	<code>ContractDao.kt</code> — query <code>WHERE isActive = 1 AND endDate >= :now</code>
Kotlin Coroutines + Flow / StateFlow	Async reactivo	<code>PaymentFormViewModel</code> — <code>stateIn(viewModelScope, ...)</code>
Navigation Component	Navegación	<code>nav_graph.xml</code> + <code>findNavController().navigate(action)</code>
ViewBinding	Acceso seguro a vistas	<code>binding.btnSave.setOnClickListener { ... }</code>
Material 3	UI	<code>MaterialAlertDialogBuilder</code> para confirmaciones de borrado

Tecnología	Para qué	Dónde verlo
MVVM + Repository	Arquitectura	ContractRepository consumido por ContractListViewModel

5. Contribución técnica de cada integrante (3:00 — 45 s c/u)

[Integrante 1] — Owner: Propiedades + Dashboard

Implementé `PropertyListFragment`, `PropertyFormFragment` y el `DashboardViewModel`. Lo más interesante fue el **Dashboard con combine de múltiples flows**: para mostrar “propiedades disponibles” combino `totalPropertyCount` y `rentedPropertyCount` para evitar la condición de carrera de leer `.value` por separado. (Mostrar el bloque `combine { t, r -> t to r }.collect`).

[Integrante 2] — Owner: Inquilinos + Contratos

Implementé el CRUD de inquilinos y contratos. La parte técnica más fuerte fue resolver la **race condition en formularios de edición**: cuando se abre un contrato existente, el flow de la entidad y el flow de las opciones del dropdown llegan en orden impredecible. Usé un campo `pendingPropertyIdToSelect` que cada collector aplica si puede o estaciona si no. (Mostrar `ContractFormFragment`). También integré la validación de **no permitir dos contratos activos** sobre la misma propiedad mediante callback `onConflict` desde el ViewModel.

[Integrante 3] — Owner: Pagos

Implementé el módulo de pagos con `PaymentDao`, `PaymentRepository` y los formularios. Técnicamente destaco dos cosas: **(1)** la validación pre-save de duplicados por (contrato, mes, año) en el ViewModel, y **(2)** el patrón **Adapter con map de etiquetas**: el `PaymentAdapter` no consulta repositorios, recibe un `Map<Long, String>` desde el ViewModel con la etiqueta “propiedad - inquilino” por contrato, actualizada con `notifyItemRangeChanged`. (Mostrar el `StateFlow<Map<Long, String>>` en `PaymentListViewModel`).

[Integrante 4] — Owner: Mantenimiento + Navegación guiada

Implementé el módulo de mantenimiento y el patrón transversal de **navegación guiada cuando faltan dependencias**. Si el usuario abre el formulario de mantenimiento sin propiedades registradas, un `MaterialAlertDialog` no cancelable lo lleva al formulario de propiedades. Para que esto funcione sin disparar diálogos falsos, los flows usan

`StateFlow<List<T>?>` con valor inicial `null` — así se distingue “cargando” de “lista vacía real”. (Mostrar el `?: return@collect` en el fragment).

6. Aprendizajes y qué haríamos diferente (0:45)

[Integrante 1]: Aprendimos a diseñar con Flows reactivos en lugar de callbacks, a usar Room con KSP, y la importancia de los estados de carga explícitos para no inducir errores de UX.

[Integrante 2]: Qué haríamos diferente:

- Adoptar **Hilt** desde el inicio en lugar de DI manual — escala mejor.
- Migrar la UI a **Jetpack Compose** para reducir XML.
- Agregar **sincronización en la nube** (Firebase / backend propio) para uso multi-dispositivo.
- Cobertura de **pruebas unitarias e instrumentadas** mucho mayor — empezamos tarde.

7. Cierre (0:15)

[Integrante 3]: Esa es **rentFlowMax** — gestión de rentas centralizada, validada y en español. Gracias, ¿preguntas?

Tips de ejecución

- Tengan la app **pre-poblada con datos demo** antes de grabar — evita capturar texto en vivo.
- Para la sección 4, dejen Android Studio abierto en split view (código + emulador).
- Cronometren un ensayo: si pasan de 10:30, recorten ejemplos de código en sección 5 a un solo snippet por persona.